

Practical No. – 1

Aim:

To perform fundamental data handling and processing operations using Python, including:

1. Parsing and extracting data from text, CSV, HTML, XML and JSON files
2. Performing read and write operations on binary files
3. Searching, splitting and replacing strings using regular expressions
4. Creating and managing a relational database using SQL CRUD operations

1. Parsing and extracting data from text, CSV, HTML, XML, and JSON files

❖ Text File

```
import re

with open("sample.txt", "r") as file:
    data = file.read()

print("Extracted Data:")
print(data)

# Find missing values
if re.search(r"Email:\s*$", data, re.MULTILINE):
    print("\nMissing Email: Found")

# Find anomalous age
ages = re.findall(r"Age:\s*(\d+)", data)

for age in ages:
    if int(age) > 100:
        print("Anomalous Age:", age)
```

Output:

```
Extracted Data:
ID: 101, Name: Rahul, Age: 21, City: Mumbai, Email: rahul@gmail.com
ID: 102, Name: Priya, Age: 22, City: Pune, Email:
ID: 103, Name: Amit, Age: 150, City: Nashik, Email: amit@gmail.com

Missing Email: Found
Anomalous Age: 150
```

❖ CSV File

```
import pandas as pd
# Read CSV file
df = pd.read_csv("sample.csv")

print("Extracted Data:")
print(df)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check anomalous age
print("\nAnomalous Age:")
print(df[df["Age"] > 100])
```

Output:

Missing Values:

```
ID      0
Name     0
Age      0
City     0
Email    1
dtype: int64
```

Anomalous Age:

```
   ID  Name  Age  City      Email
2  103  Amit  150  Nashik  amit@gmail.com
```

❖ HTML File

```
from bs4 import BeautifulSoup
import pandas as pd

# Read HTML file
with open("sample.html", "r") as file:
    soup = BeautifulSoup(file, "html.parser")

# Extract table data
table = soup.find("table")
df = pd.read_html(str(table))[0]

print("Extracted Data:")
print(df)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check anomalous age
print("\nAnomalous Age:")
print(df[df["Age"] > 100])
```

Output:

Extracted Data:

```
   ID  Name  Age  City      Email
0  101  Rahul  21  Mumbai  rahul@gmail.com
1  102  Priya  22   Pune         NaN
2  103   Amit  150  Nashik  amit@gmail.com
```

Missing Values:

```
ID      0
Name     0
Age      0
City     0
Email    1
dtype: int64
```

Anomalous Age:

```
   ID  Name  Age  City      Email
2  103  Amit  150  Nashik  amit@gmail.com
```

❖ XML File

```
import xml.etree.ElementTree as ET

# Read XML file
tree = ET.parse("sample.xml")
root = tree.getroot()

print("Extracted Data:")

for student in root.findall("student"):
    print(
        student.find("ID").text,
        student.find("Name").text,
        student.find("Age").text,
        student.find("City").text,
        student.find("Email").text
    )

# Check missing values
print("\nMissing Values:")

for student in root.findall("student"):
    if not student.find("Email").text:
        print("Missing Email found for:", student.find("Name").text)

# Check anomalous age
print("\nAnomalous Age:")

for student in root.findall("student"):
    age = int(student.find("Age").text)
    if age > 100:
        print("Anomalous Age:", age)
```

Output:

```
Extracted Data:
101 Rahul 21 Mumbai rahul@gmail.com
102 Priya 22 Pune None
103 Amit 150 Nashik amit@gmail.com

Missing Values:
Missing Email found for: Priya

Anomalous Age:
Anomalous Age: 150
```

❖ JSON File

```
import json

# Read JSON file
with open("sample.json", "r") as file:
    data = json.load(file)

print("Extracted Data:")

for student in data["students"]:
    print(student)

# Check missing values
print("\nMissing Values:")

for student in data["students"]:
    for key, value in student.items():
        if value == "":
            print("Missing", key, "for:", student["Name"])

# Check anomalous age
print("\nAnomalous Age:")

for student in data["students"]:
    if student["Age"] > 100:
        print("Anomalous Age:", student["Age"])
```

Output:

```
Extracted Data:
{'ID': 101, 'Name': 'Rahul', 'Age': 21, 'City': 'Mumbai', 'Email': 'rahul@gmail.com'}
{'ID': 102, 'Name': 'Priya', 'Age': 22, 'City': 'Pune', 'Email': ''}
{'ID': 103, 'Name': 'Amit', 'Age': 150, 'City': 'Nashik', 'Email': 'amit@gmail.com'}

Missing Values:
Missing Email for: Priya

Anomalous Age:
Anomalous Age: 150
```

2. Performing read and write operations on binary files

```
# Writing to a binary file
data = b"Hello, Data Engineering!"
with open("sample.bin", "wb") as file:
    file.write(data)

print("Data written successfully.")

# Reading from a binary file
with open("sample.bin", "rb") as file:
    data = file.read()

print("Data read from binary file:")
print(data)
```

Output:

```
Data written successfully.  
Data read from binary file:  
b'Hello, Data Engineering!'
```

3. Searching, splitting and replacing strings using regular expressions

```
import re  
  
text = "Data Engineering is useful. Data Science is interesting."  
  
# Search  
search = re.search("Engineering", text)  
print("Search Result:", search.group())  
  
# Split  
words = re.split(" ", text)  
print("\nSplit Result:", words)  
  
# Replace  
new_text = re.sub("Data", "Big Data", text)  
print("\nReplaced Text:", new_text)
```

Output:

```
Search Result: Engineering  
  
Split Result: ['Data', 'Engineering', 'is', 'useful.', 'Data', 'Science', 'is', 'interesting.']  
  
Replaced Text: Big Data Engineering is useful. Big Data Science is interesting.
```

4. Creating and managing a relational database using SQL CRUD operations

```
import sqlite3  
  
# Connect to database  
conn = sqlite3.connect("student.db")  
cursor = conn.cursor()  
  
# CREATE  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS students (  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    age INTEGER,  
    city TEXT  
)  
""")  
  
# INSERT  
cursor.execute("INSERT INTO students VALUES (1, 'Rahul', 21, 'Mumbai')")  
cursor.execute("INSERT INTO students VALUES (2, 'Priya', 22, 'Pune')")  
conn.commit()  
  
cursor.execute("UPDATE students SET city='Nashik' WHERE id=2")  
conn.commit()
```

```
# READ
print("Student Records:")
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# UPDATE
cursor.execute("UPDATE students SET city='Nashik' WHERE id=2")
conn.commit()

# DELETE
cursor.execute("DELETE FROM students WHERE id=1")
conn.commit()

print("\nRecords after UPDATE and DELETE:")
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

conn.close()
```

Output:

```
Student Records:
[(1, 'Rahul', 21, 'Mumbai'), (2, 'Priya', 22, 'Pune')]

Records after UPDATE and DELETE:
[(2, 'Priya', 22, 'Nashik')]
```

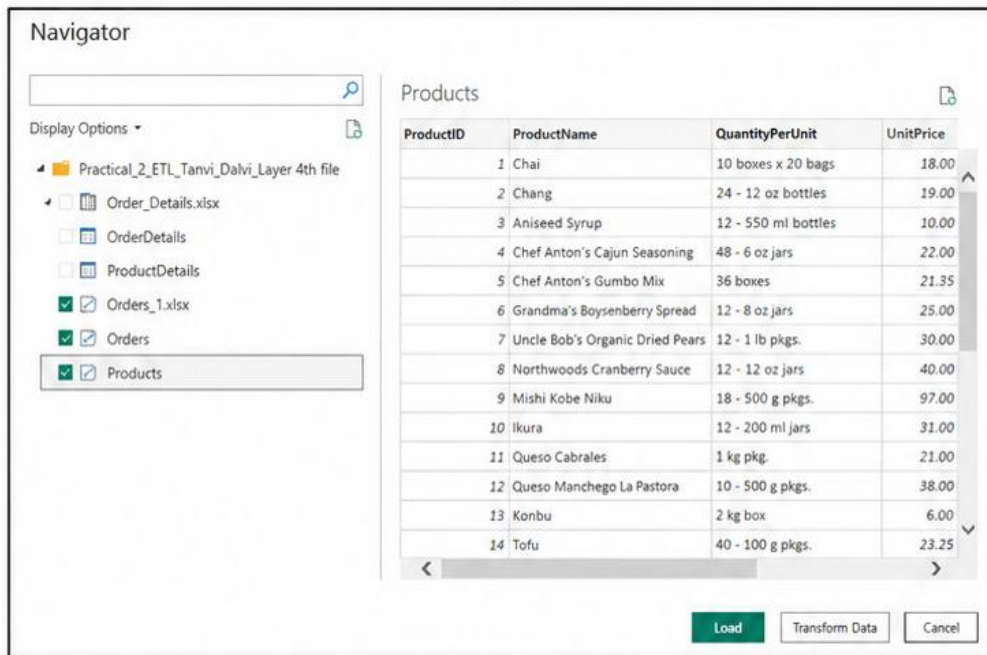
Practical No. – 2

Aim:

To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.

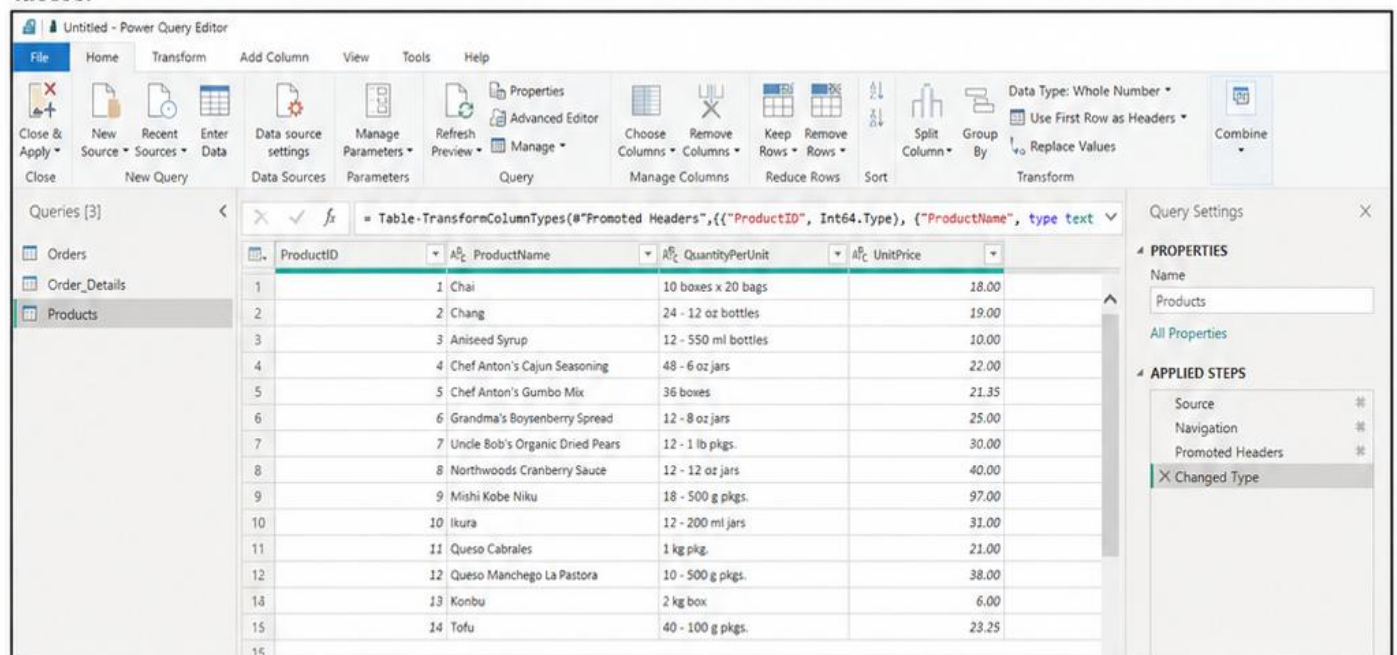
Step 1 — Import Excel Data into Power BI

The Excel workbook containing the Products, Orders, and Order_Details tables was imported into Power BI for further data transformation and analysis.



Step 2 — Open Power Query Editor

Power Query Editor was opened to perform the required data transformation operations on the imported tables.



Step 3 — Remove Unnecessary Columns

Unnecessary columns were removed from the Products table to retain only the fields required for further analysis.

Queries [6] < fx = Table.SelectRows("#Changed Type", each [UnitPrice] > 0) Query Settings ×

PROPERTIES

Name
Products

All Properties

APPLIED STEPS

Source ⚙
Navigation ⚙
Promoted Headers ⚙
X Changed Type

	ProductID	ProductName	QuantityPerUnit	UnitsInStock
1	1	Chai	10 boxes x 20 bags	18
2	2	Chang	24 - 12 oz bottles	19
3	3	Aniseed Syrup	12 - 550 ml bottles	10
4	4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22
5	5	Chef Anton's Gumbo Mix	36 boxes	21.35
6	6	Grandma's Boysenberry Spread	12 - 8 oz jars	25
7	7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30
8	8	Northwoods Cranberry Sauce	12 - 12 oz jars	40
9	9	Mishi Kobe Niku	18 - 500 g pkgs.	97
10	10	Ikura	12 - 200 ml jars	31
11	11	Queso Cabrales	1 kg pkg.	21
12	12	Queso Manchego La Pastora	10 - 500 g pkgs.	38
13	13	Konbu	2 kg box	6
14	14	Tofu	40 - 100 g pkgs.	23.25
15	15	Genen Shouyu	24 - 250 ml bottles	15.5

Step 4 — Verify the Data Type of UnitsInStock

The data type of the UnitsInStock column was verified as Whole Number, which is appropriate for representing stock quantities.

123 UnitsInStock

Decimal Number
Fixed decimal number
✓ Whole Number
Percentage
Date/Time
Date
Time
Date/Time/Timezone
Duration
Text
True/False
Binary
Using Locale...

Number (decimal)
Number (fixed decimal)
✓ Whole Number
Percentage
Date/Time
Date
Time
Date/Time/Timezone
Duration
Text
True/False
Binary

Step 5 — Calculate Line Total

A new LineTotal column was created by multiplying UnitPrice by Quantity for each order detail.

Queries [3] < fx = Table.AddColumn("#Changed Type", "LineTotal", each [UnitPrice] * [Quantity])

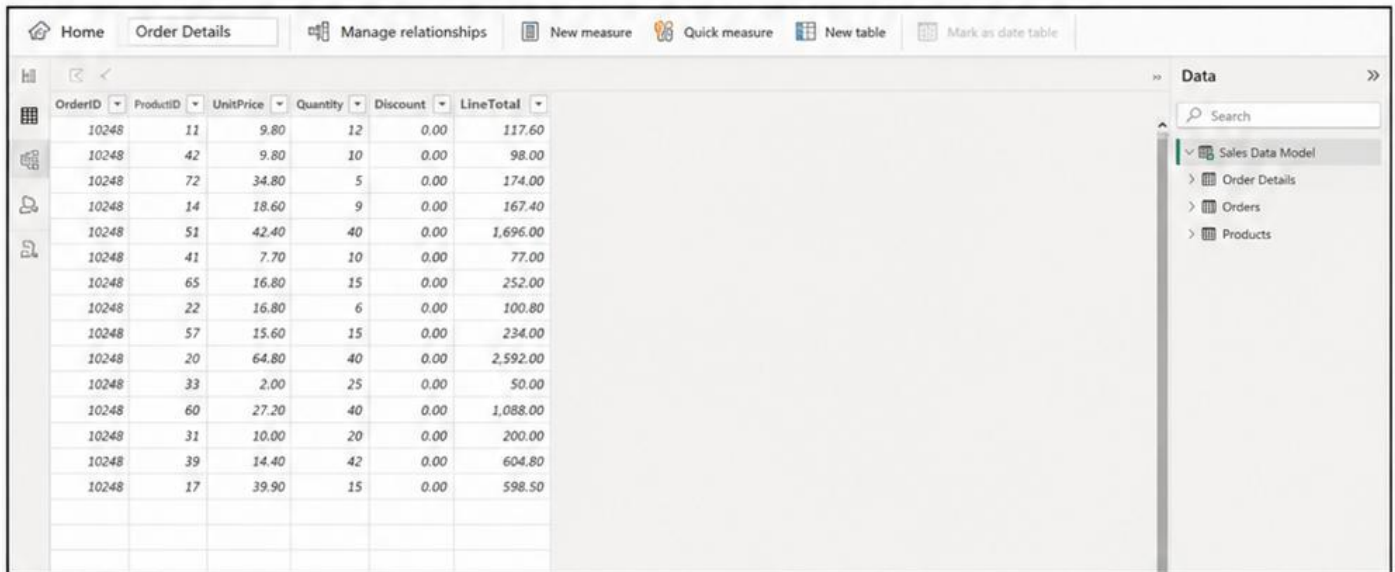
	OrderID	ProductID	Quantity	UnitPrice	LineTotal
1	10248	11	12	9.80	117.60
2	10248	42	10	9.80	98.00
3	10248	72	5	34.80	174.00
4	10248	14	9	18.60	167.40
5	10248	51	40	42.40	1,696.00
6	10248	41	10	7.70	77.00
7	10248	65	15	16.80	252.00
8	10248	22	6	16.80	100.80
9	10248	57	15	15.60	234.00
10	10248	20	40	64.80	2,592.00
11	10248	33	25	2.00	50.00
12	10248	60	40	27.20	1,088.00
13	10248	31	20	10.00	200.00
14	10248	39	42	14.40	604.80
15	10248	17	15	39.90	598.50

Step 6 — Rename and Reorder Columns

The columns were verified and arranged in a suitable order for further analysis.

Step 7 — Load the Transformed Data

The transformed data was loaded into Power BI Desktop using the Close & Apply option.

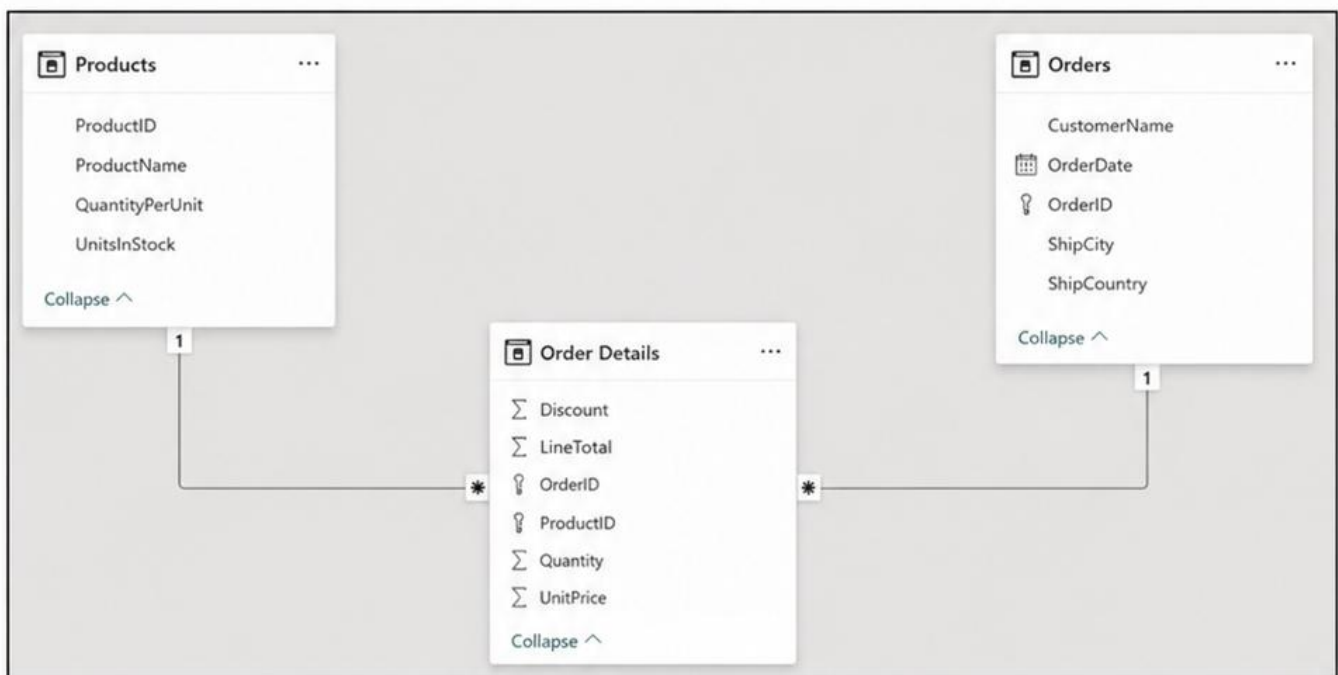


The screenshot shows the Power BI Desktop interface. The 'Home' ribbon is active, and the 'Order Details' table is selected in the 'Data' pane on the right. The table is displayed in the main view area, showing columns: OrderID, ProductID, UnitPrice, Quantity, Discount, and LineTotal. The data is as follows:

OrderID	ProductID	UnitPrice	Quantity	Discount	LineTotal
10248	11	9.80	12	0.00	117.60
10248	42	9.80	10	0.00	98.00
10248	72	34.80	5	0.00	174.00
10248	14	18.60	9	0.00	167.40
10248	51	42.40	40	0.00	1,696.00
10248	41	7.70	10	0.00	77.00
10248	65	16.80	15	0.00	252.00
10248	22	16.80	6	0.00	100.80
10248	57	15.60	15	0.00	234.00
10248	20	64.80	40	0.00	2,592.00
10248	33	2.00	25	0.00	50.00
10248	60	27.20	40	0.00	1,088.00
10248	31	10.00	20	0.00	200.00
10248	39	14.40	42	0.00	604.80
10248	17	39.90	15	0.00	598.50

Step 8 — Create Relationships Between Tables

Relationships were created between the related tables using their common key fields to establish the data model.



Step 9 — Result

The ETL process was successfully performed in Power BI by extracting data from an Excel workbook, transforming the data using Power Query, creating the LineTotal column, loading the transformed data, and establishing relationships between the tables.

Practical No. – 3

Aim:

To implement and demonstrate single and multiple document insertion and retrieval operations in MongoDB.

Step 1: Insert Single Document

Query:

```
db.items.insertOne({ name: "laptop", price: 999 });
```

Output:

```
Enterprise practical3> db.items.insertOne({ name: "laptop", price: 999 });
{
  acknowledged: true,
  insertedId: ObjectId('6a858d72abb1be9fedc2c804')
}
```

Step 2: View Inserted Document

Query:

```
db.items.find();
```

Output:

```
Enterprise practical3> db.items.find();
[
  {
    _id: ObjectId('6a858e3aabb1be9fedc2c805'),
    name: 'laptop',
    price: 999
  }
]
```

Step 3: Insert Multiple Documents

Query:

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a858ec3abb1be9fedc2c806'),
    '1': ObjectId('6a858ec3abb1be9fedc2c807'),
    '2': ObjectId('6a858ec3abb1be9fedc2c808')
  }
}
```

Step 4: View Products in Formatted JSON

Query:

```
db.products.find().pretty();
```

Output:

```
Enterprise practical3> db.products.find().pretty()
[
  {
    _id: ObjectId('6a858f8aabb1be9fedc2c809'),
    name: 'phone',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('6a858f8aabb1be9fedc2c80a'),
    name: 'tablet',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6a858f8aabb1be9fedc2c80b'),
    name: 'watch',
    price: 150,
    stock: 0
  }
]
```

Practical No. – 4

Aim:

To perform noise elimination, feature selection, and exploratory data analysis (EDA) using Python libraries.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# Sample Dataset
data = {
    'Age': [20, 21, 22, 23, 24, 100], # 100 is an outlier (noise)
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1] # Constant feature
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# 1. Noise Elimination (Outlier Removal)
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]

print("\nDataset after Noise Elimination:")
print(df_clean)

# 2. Feature Selection (Remove constant or low-variance features)
selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selected.get_support()]

print("\nSelected Features:")
print(selected_columns)

# 3. Exploratory Data Analysis (EDA)
```

```

print("\nDataset Information:")
print(df_clean.info())

print("\nStatistical Summary:")
print(df_clean.describe())

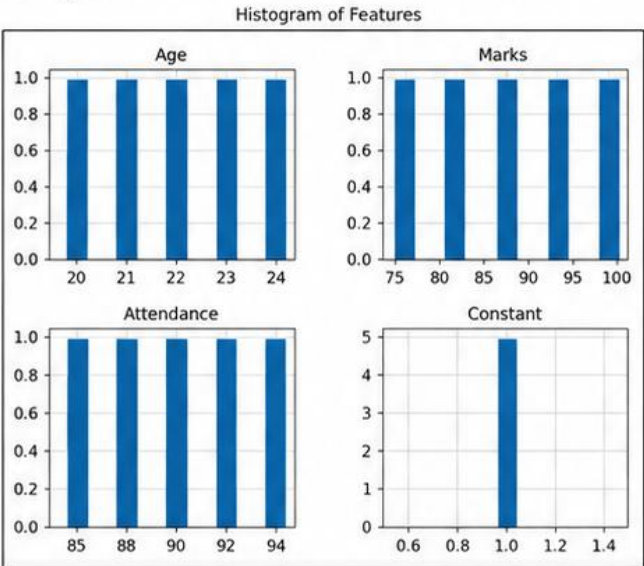
print("\nCorrelation Matrix:")
print(df_clean.corr())

# Histogram
df_clean.hist(figsize=(8, 6))
plt.suptitle("Histogram of Features")
plt.show()

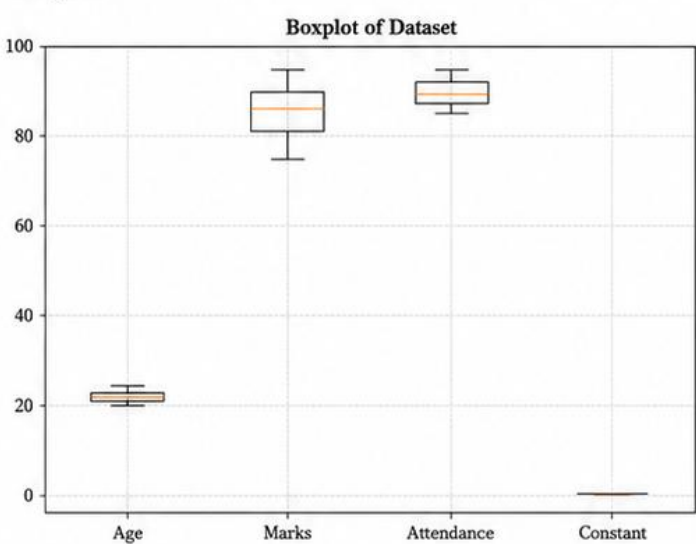
# Boxplot
df_clean.boxplot(figsize=(8, 5))
plt.title("Boxplot of Dataset")
plt.show()

```

Histogram:



Boxplot:



```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Age          5 non-null      int64
1   Marks        5 non-null      int64
2   Attendance   5 non-null      int64
3   Constant     5 non-null      int64
dtypes: int64(4)
memory usage: 292.0 bytes

Dataset after Noise Elimination:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Age          4 non-null      int64
1   Marks        4 non-null      int64
2   Attendance   4 non-null      int64
3   Constant     4 non-null      int64
dtypes: int64(4)
memory usage: 260.0 bytes

```

Statistical Summary:

	Age	Marks	Attendance	Constant
count	4.000000	4.000000	4.000000	4.0
mean	22.000000	85.000000	89.000000	1.0
std	1.290994	7.071068	3.651484	0.0
min	20.000000	75.000000	85.000000	1.0
25%	21.500000	80.000000	87.750000	1.0
50%	22.000000	85.000000	90.000000	1.0
75%	22.500000	90.000000	91.250000	1.0
max	24.000000	95.000000	94.000000	1.0

Correlation Matrix:

	Age	Marks	Attendance	Constant
Age	1.000000	1.000000	0.995893	NaN
Marks	1.000000	1.000000	0.995893	NaN
Attendance	0.995893	0.995893	1.000000	NaN
Constant	NaN	NaN	NaN	NaN

Practical No. - 5

Aim:

To extract user data from a REST API and a flat CSV file, transform and merge the data using Python.

Code:

```
import pandas as pd
import requests

def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        # Normalizing JSON data into a flat table
        return pd.json_normalize(data)
    except requests.exceptions.RequestException as e:
        print(f'API Error: {e}')
        return pd.DataFrame()

def extract_csv_data(file_path):
    """Reads a flat CSV file."""
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f'File Error: {e}')
        return pd.DataFrame()

if __name__ == "__main__":
    # 1. API Extraction
    api_url = "https://jsonplaceholder.typicode.com/users"
    api_df = extract_api_data(api_url)
    api_df = api_df[["id", "name", "email", "company.name"]]
    api_df.rename(columns={"company.name": "company"}, inplace=True)

    # 2. Flat File Extraction (Mock CSV Data)
    # Assume 'locations.csv' has columns: id, city, country
    csv_file = "locations.csv"

    # Creating a dummy file for the lab test run
    pd.DataFrame(
        {
            "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            "city": [
```

```

        "New York",
        "London",
        "Paris",
        "Tokyo",
        "Berlin",
        "Delhi",
        "Sydney",
        "Moscow",
        "Cairo",
        "Beijing",
    ],
}
).to_csv(csv_file, index=False)

csv_df = extract_csv_data(csv_file)

# 3. Data Transformation & Merging
if not api_df.empty and not csv_df.empty:
    merged_df = pd.merge(api_df, csv_df, on="id", how="inner")
    print("\n--- Merged ETL Pipeline Data View ---")
    print(merged_df.head())

# Save to target Data Lake / Warehouse stage
merged_df.to_csv("cleaned_warehouse_profiles.csv", index=False)
print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")

```

Output:

```

--- Merged ETL Pipeline Data View ---
   id  name                email                company    city
0   1  Leanne Graham    Sincere@april.biz    Romaguera-Crona  New York
1   2   Ervin Howell    Shanna@melissa.tv    Deckow-Crist    London
2   3  Clementine Bauch    Nathan@yesenia.net  Romaguera-Jacobson  Paris
3   4  Patricia Lebsack  Julianne.OConner@kory.org    Robel-Corkery    Tokyo
4   5  Chelsey Dietrich  Lucio_Hettinger@annie.ca    Keebler LLC      Berlin

Data successfully saved to 'cleaned_warehouse_profiles.csv'

```

Practical No. – 6

Aim:

To design and configure a scheduled Apache Airflow DAG for orchestrating an ETL workflow through sequential tasks for pipeline initialization, data extraction, and successful pipeline completion.

Code:

```
!pip install apache-airflow -q

import os
import subprocess

os.environ["AIRFLOW_HOME"] = "/content/airflow"
os.environ["AIRFLOW__CORE__DAGS_FOLDER"] = "/content/airflow/dags"

os.makedirs("/content/airflow/dags", exist_ok=True)

# Create extraction script
with open("/content/airflow/data_extraction.py", "w") as f:
    f.write("""
print("Starting data extraction...")
print("Extracting data from source...")
print("Data extraction completed successfully.")
""")

# Create Airflow DAG
with open("/content/airflow/dags/data_pipeline_dag.py", "w") as f:
    f.write("""
from datetime import datetime, timedelta

from airflow.sdk import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="ETL pipeline orchestration",
    schedule="@daily",
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(
        task_id="start pipeline"
```

```

)

execute_extraction = BashOperator(
    task_id="run_extraction_script",
    bash_command="python /content/airflow/data_extraction.py"
)

pipeline_complete = BashOperator(
    task_id="log_pipeline_success",
    bash_command='echo "ETL Execution completed successfully"'
)

start_pipeline >> execute_extraction >> pipeline_complete
"""

print("DAG created successfully.")
print("DAG Name: university_etl_orchestration")
print("Schedule: Daily")
print()

# Initialize Airflow database
subprocess.run(
    ["airflow", "db", "migrate"],
    env=os.environ
)

print("\nRunning DAG tasks...\n")

# Run complete DAG
result = subprocess.run(
    [
        "airflow",
        "dags",
        "test",
        "university_etl_orchestration",
        "2026-08-20"
    ],
    env=os.environ,
    capture_output=True,
    text=True
)

print(result.stdout)

if result.stderr:
    print(result.stderr)

print("\nETL Pipeline Practical Completed!")

```

Output:

```
DAG created successfully.
DAG Name: university_etl_orchestration
Schedule: Daily
```

Running DAG tasks...

```
2026-08-20T03:24:55.420209Z [info] ] DAG bundles loaded: dags-folder, example_dags, apache-airflow-providers-common-sql-example-dags, apache-airflow
2026-08-20T03:24:55.422695Z [info] ] Added new DAG bundle dags-folder to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc
2026-08-20T03:24:55.423199Z [info] ] Added new DAG bundle example_dags to the database [airflow.dag_processing.bundles.manager.DagBundlesManager] loc
2026-08-20T03:24:55.423543Z [info] ] Added new DAG bundle apache-airflow-providers-common-sql-example-dags to the database [airflow.dag_processing.b
2026-08-20T03:24:55.423853Z [info] ] Added new DAG bundle apache-airflow-providers-standard-example-dags to the database [airflow.dag_processing.bu
2026-08-20T03:24:55.433208Z [info] ] Filling up the DagBag from /content/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:464
2026-08-20T03:24:55.458111Z [info] ] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=1 loc=dag.py:210
2026-08-20T03:24:55.474187Z [info] ] Creating ORM DAG for university_etl_orchestration [airflow.dag_processing.collection] loc=collection.py:102
2026-08-20T03:24:55.492923Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:
2026-08-20T03:24:55.493320Z [info] ] setting next_dagrun_info [airflow.models.dag] loc=dag.py:848 next_dagrun='2026-08-20 00:00:00+00:00' next
2026-08-20T03:24:55.570418Z [info] ] DAG bundles loaded: dags-folder, example_dags, apache-airflow-providers-common-sql-example-dags, apache-airflow
2026-08-20T03:24:55.578863Z [info] ] Filling up the DagBag from /content/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:464
2026-08-20T03:24:55.584888Z [info] ] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=1 loc=dag.py:210
2026-08-20T03:24:55.594570Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:
2026-08-20T03:24:55.594866Z [info] ] setting next_dagrun_info [airflow.models.dag] loc=dag.py:848 next_dagrun='2026-08-20 00:00:00+00:00' next
2026-08-20T03:24:55.624062Z [info] ] creating dag run [airflow.serialization.definitions.dag] loc=dag.py:607 logical_date=DateTime(20
```

```
Previous state of the Task instance: TaskInstanceState.QUEUED
Current task name:start_pipeline
Dag name:university_etl_orchestration
2026-08-20T03:25:02.825713Z [info] ] ::endgroup:: [task] loc=task_runner.py:2232
2026-08-20T03:25:02.825956Z [info] ] ::group::Post Execute [task] loc=task_runner.py:1580
2026-08-20T03:25:02.850195Z [info] ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(EmptyOperator): start_pipeline>
2026-08-20T03:25:02.861369Z [info] ] ::endgroup:: [task] loc=task_runner.py:2379
2026-08-20T03:25:02.880112Z [info] ] [DAG TEST] end task task_id=start_pipeline map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1563
2026-08-20T03:25:02.894454Z [info] ] [DAG TEST] starting task_id=run_extraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1501
2026-08-20T03:25:02.895190Z [info] ] [DAG TEST] running task <TaskInstance: university_etl_orchestration.run_extraction_script manual_2026-08-20T03:
2026-08-20T03:25:02.919756Z [info] ] Task started [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
2026-08-20T03:25:02.921399Z [info] ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
2026-08-20T03:25:02.948583Z [info] ] Updating RenderedTaskInstanceFields [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=0:
```

```
Task instance is in running state
Previous state of the Task instance: TaskInstanceState.QUEUED
Current task name:run_extraction_script
Dag name:university_etl_orchestration
2026-08-20T03:25:02.968760Z [info] ] ::endgroup:: [task] loc=task_runner.py:2232
2026-08-20T03:25:02.969514Z [info] ] Tmp dir root location: /tmp [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:02.969967Z [info] ] Running command: ['/usr/bin/bash', '-c', 'python /content/airflow/data_extraction.py'] [airflow.task.hooks.air
2026-08-20T03:25:02.970692Z [info] ] Output: [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:03.004779Z [info] ] Starting data extraction... [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:03.005034Z [info] ] Extracting data from source... [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:03.005150Z [info] ] Data extraction completed successfully. [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.Subpro
2026-08-20T03:25:03.009726Z [info] ] Command exited with return code 0 [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHoc
2026-08-20T03:25:03.010299Z [info] ] ::group::Post Execute [task] loc=task_runner.py:1580
2026-08-20T03:25:03.010546Z [info] ] Pushing xcom [task] loc=task_runner.py:2280 ti=RuntimeTaskInstance(id=UUID('01a01d33-49da-71:
2026-08-20T03:25:03.052063Z [info] ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(BashOperator): run_extraction_script>
2026-08-20T03:25:03.062663Z [info] ] ::endgroup:: [task] loc=task_runner.py:2379
2026-08-20T03:25:03.077144Z [info] ] [DAG TEST] end task task_id=run_extraction_script map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1563
2026-08-20T03:25:03.091813Z [info] ] [DAG TEST] starting task_id=log_pipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1501
2026-08-20T03:25:03.092187Z [info] ] [DAG TEST] running task <TaskInstance: university_etl_orchestration.log_pipeline_success manual_2026-08-20T03:
2026-08-20T03:25:03.112020Z [info] ] Task started [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
2026-08-20T03:25:03.113424Z [info] ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
2026-08-20T03:25:03.138557Z [info] ] Updating RenderedTaskInstanceFields [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=0:
Task instance is in running state
```

```
Current task name:log_pipeline_success
Dag name:university_etl_orchestration
2026-08-20T03:25:03.153844Z [info] ] ::endgroup:: [task] loc=task_runner.py:2232
2026-08-20T03:25:03.154538Z [info] ] Tmp dir root location: /tmp [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:03.154895Z [info] ] Running command: ['/usr/bin/bash', '-c', 'echo "ETL Execution completed successfully"'] [airflow.task.hooks.air
2026-08-20T03:25:03.155530Z [info] ] Output: [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook]
2026-08-20T03:25:03.156676Z [info] ] ETL Execution completed successfully [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.Subprocess
2026-08-20T03:25:03.156839Z [info] ] Command exited with return code 0 [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHoc
2026-08-20T03:25:03.157105Z [info] ] ::group::Post Execute [task] loc=task_runner.py:1580
2026-08-20T03:25:03.157299Z [info] ] Pushing xcom [task] loc=task_runner.py:2280 ti=RuntimeTaskInstance(id=UUID('01a01d33-49db-78:
2026-08-20T03:25:03.189099Z [info] ] Task instance state updated [airflow.api_fastapi.execution_api.routes.task_instances] correlation_id=01a01d:
Task instance in success state
Previous state of the Task instance: TaskInstanceState.RUNNING
Task operator:<Task(BashOperator): log_pipeline_success>
2026-08-20T03:25:03.198872Z [info] ] ::endgroup:: [task] loc=task_runner.py:2379
2026-08-20T03:25:03.209586Z [info] ] [DAG TEST] end task task_id=log_pipeline_success map_index=-1 [airflow.sdk.definitions.dag] loc=dag.py:1563
2026-08-20T03:25:03.212618Z [info] ] Marking run <DagRun university_etl_orchestration @ 2026-08-20 00:00:00+00:00: manual_2026-08-20T03:24:55.5690
Dag run in success state
Dag run start:2026-08-20 00:00:00+00:00 end:2026-08-20 03:25:03.212826+00:00
2026-08-20T03:25:03.213718Z [info] ] DagRun Finished: dag_id=university_etl_orchestration, logical_date=2026-08-20 00:00:00+00:00, run_id>manual_20
```

ETL Pipeline Practical Completed!

Practical No. - 7

Aim:

To design and implement an ETL pipeline using Python for extracting data from CSV files, cleaning and transforming the data, validating the records, and loading the processed data into a database.

Code:

```
import pandas as pd
import sqlite3

print("===== PRACTICAL 7: ETL PIPELINE =====")

# 1. CSV ETL
print("\n--- 1. CSV ETL Pipeline ---")
df = pd.DataFrame({"ID": [1, 2, 3], "Name": ["Amit", "Neha", "Rahul"],
                  "Age": [22, None, 25], "Salary": [30000, 40000, 35000]})
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] *= 1.10
con = sqlite3.connect("etl.db")
df.to_sql("employees", con, if_exists="replace", index=False)
con.close()
print(df)

# 2. Multiple CSV Files
print("\n--- 2. Combining Multiple CSV Files ---")
a = pd.DataFrame({"ID": [1, 2], "Name": ["Amit", "Neha"]})
b = pd.DataFrame({"ID": [3, 4], "Name": ["Rahul", "Priya"]})
combined = pd.concat([a, b], ignore_index=True)
print(combined)

# 3. JSON ETL
print("\n--- 3. JSON ETL Pipeline ---")
df = pd.DataFrame({"ID": [1, 2, 3], "Name": ["Amit", "Neha", "Rahul"], "Age": [22, 25, 28]})
df["Age_Group"] = df["Age"].apply(lambda x: "Young" if x < 25 else "Adult")
con = sqlite3.connect("json.db")
df.to_sql("users", con, if_exists="replace", index=False)
con.close()
print(df)

# 4. Remove Invalid Records
print("\n--- 4. Invalid Record Removal ---")
df = pd.DataFrame({"ID": [1, 2, 3], "Age": [22, -5, 150]})
print("Original Records:")
print(df)
df = df[(df["Age"] >= 0) & (df["Age"] <= 100)]
print("\nAfter Removing Invalid Records:")
print(df)

# 5. Data Validation
print("\n--- 5. Data Validation ---")
df = pd.DataFrame({"ID": [1, 2, 3], "Age": [22, 25, 150]})
valid = df[df["Age"].between(18, 100)]
con = sqlite3.connect("validated.db")
```

```

valid.to_sql("employees", con, if_exists="replace", index=False)
con.close()
print("Validated Records:")
print(valid)

# 6. Incremental Loading
print("\n--- 6. Incremental Data Loading ---")
df = pd.DataFrame({"ID": [1, 2, 3], "Name": ["Amit", "Neha", "Rahul"]})
con = sqlite3.connect("incremental.db")
old = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", con)
if old.empty:
    df.to_sql("employees", con, index=False)
    new = df
else:
    new = pd.DataFrame()
print("New Records Loaded:")
print(new)
con.close()

print("\n===== ALL ETL OPERATIONS COMPLETED =====")

```

Output:

===== PRACTICAL 7: ETL PIPELINE =====

--- 1. CSV ETL Pipeline ---

	ID	Name	Age	Salary
0	1	Amit	22.0	33000.0
1	2	Neha	26.5	44000.0
2	3	Rahul	25.0	38500.0
3	4	Priya	28.0	49500.0

Data loaded into database successfully.

--- 2. Combining Multiple CSV Files ---

	ID	Name	Department
0	1	Amit	IT
1	2	Neha	HR
2	3	Rahul	Sales
3	4	Priya	IT
4	5	Ankit	Finance
5	6	Meena	HR

CSV files combined successfully.

--- 3. JSON ETL Pipeline ---

	ID	Name	Age	Age_Group
0	1	Amit	22	Young
1	2	Neha	25	Adult
2	3	Rahul	28	Adult

JSON data loaded successfully.

--- 4. Invalid Record Removal ---

Original Records:

	ID	Name	Age
0	1	Amit	22
1	2	Neha	-5
2	3	Rahul	25
3	4	Priya	150
4	5	Ankit	28

After Removing Invalid Records:

	ID	Name	Age
0	1	Amit	22
2	3	Rahul	25
4	5	Ankit	28

--- 5. Data Validation ---

Validated Records:

	ID	Name	Age
0	1	Amit	22
1	2	Neha	25
3	4	Priya	28

Validated data loaded successfully.

--- 6. Incremental Data Loading ---

New Records Loaded:

Empty DataFrame

Columns: [ID, Name, Salary]

Index: []

===== ALL ETL OPERATIONS COMPLETED =====

Practical No. - 8

Aim:

To implement PySpark DataFrame operations for reading and analyzing CSV data, including schema inspection, filtering, grouping, aggregation, duplicate removal, joining datasets, and calculating average sales by product category.

Code:

```
pip show pyspark

from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, sum

spark = SparkSession.builder.appName("Practical8").getOrCreate()

# Create sample sales data
data = [
    (1, "Laptop", "Electronics", 50000),
    (2, "Phone", "Electronics", 30000),
    (3, "Chair", "Furniture", 5000),
    (4, "Table", "Furniture", 8000),
    (5, "Laptop", "Electronics", 50000),
    (5, "Laptop", "Electronics", 50000)
]

columns = ["ID", "Product", "Category", "Sales"]

df = spark.createDataFrame(data, columns)

print("\n--- 1. Schema ---")
df.printSchema()

print("\n--- 2. Filtering ---")
df.filter(df.Sales > 10000).show()

print("\n--- 3. Grouping and Aggregation ---")
df.groupBy("Category").agg(sum("Sales").alias("Total_Sales")).show()

print("\n--- 4. Remove Duplicate Records ---")
df.dropDuplicates().show()

# Second dataset for join
customer_data = [
    (1, "Amit"),
    (2, "Neha"),
    (3, "Rahul"),
    (4, "Priya"),
    (5, "Ankit")
]

customers = spark.createDataFrame(customer_data, ["ID", "Customer"])
```

```

print("\n--- 5. Join ---")
df.join(customers, "ID", "inner").show()

print("\n--- 6. Average Sales by Product Category ---")
df.groupBy("Category").agg(
    avg("Sales").alias("Average_Sales")
).show()

spark.stop()

```

Output:

--- 1. Schema ---

```

root
 |-- ID: long (nullable = true)
 |-- Product: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sales: long (nullable = true)

```

--- 2. Filtering ---

ID	Product	Category	Sales
1	Laptop	Electronics	50000
2	Phone	Electronics	30000
5	Laptop	Electronics	50000
5	Laptop	Electronics	50000

--- 3. Grouping and Aggregation ---

Category	Total_Sales
Electronics	180000
Furniture	13000

--- 4. Remove Duplicate Records ---

ID	Product	Category	Sales
3	Chair	Furniture	5000
1	Laptop	Electronics	50000
2	Phone	Electronics	30000
4	Table	Furniture	8000
5	Laptop	Electronics	50000

--- 5. Join ---

ID	Product	Category	Sales	Customer
1	Laptop	Electronics	50000	Amit
2	Phone	Electronics	30000	Neha
3	Chair	Furniture	5000	Rahul
4	Table	Furniture	8000	Priya
5	Laptop	Electronics	50000	Ankit
5	Laptop	Electronics	50000	Ankit

--- 6. Average Sales by Product Category ---

Category	Average_Sales
Electronics	45000.0
Furniture	6500.0

Practical No. – 9

Aim:

To design and implement an end-to-end data pipeline that extracts e-commerce data from CSV files, cleans and transforms the data using Python/Pandas, loads it into a SQL database, generates a report, and handles both historical and newly arriving data.

Code:

```
import pandas as pd
import sqlite3

print("===== PRACTICAL 9: END-TO-END ETL PIPELINE =====")

# 1. Historical E-Commerce Data
print("\n--- 1. Historical Data ---")

customers = pd.DataFrame({
    "CustomerID": [1, 2, 3],
    "Name": ["Amit", "Neha", "Rahul"]
})

products = pd.DataFrame({
    "ProductID": [101, 102, 103],
    "Product": ["Laptop", "Phone", "Headphones"],
    "Price": [50000, 30000, 2000]
})

orders = pd.DataFrame({
    "OrderID": [1, 2, 3],
    "CustomerID": [1, 2, 3],
    "ProductID": [101, 102, 103],
    "Quantity": [1, 2, 3]
})

payments = pd.DataFrame({
    "PaymentID": [1, 2, 3],
    "OrderID": [1, 2, 3],
    "Amount": [50000, 60000, 6000]
})

print("Customers:")
print(customers)
print("\nProducts:")
print(products)
print("\nOrders:")
print(orders)
print("\nPayments:")
print(payments)

# 2. Data Cleaning and Transformation
print("\n--- 2. Data Cleaning and Transformation ---")

customers.drop_duplicates(inplace=True)
```

```

orders.dropna(inplace=True)

orders["Total"] = orders["Quantity"] * orders["ProductID"].map(
    products.set_index("ProductID")["Price"]
)

print(orders)

# 3. Newly Arriving Data
print("\n--- 3. Newly Arriving Data ---")

new_orders = pd.DataFrame({
    "OrderID": [4],
    "CustomerID": [1],
    "ProductID": [102],
    "Quantity": [1]
})

new_orders["Total"] = new_orders["Quantity"] * new_orders["ProductID"].map(
    products.set_index("ProductID")["Price"]
)

print(new_orders)

# 4. Historical + New Data
print("\n--- 4. Combined Data ---")

orders = pd.concat([orders, new_orders], ignore_index=True)

print(orders)

# 5. Load into SQL Database
print("\n--- 5. SQL Database Loading ---")

conn = sqlite3.connect("ecommerce.db")

customers.to_sql("customers", conn, if_exists="replace", index=False)
products.to_sql("products", conn, if_exists="replace", index=False)
orders.to_sql("orders", conn, if_exists="replace", index=False)
payments.to_sql("payments", conn, if_exists="replace", index=False)

print("Customers, Products, Orders and Payments loaded successfully.")

# 6. Generate Report
print("\n--- 6. Final E-Commerce Report ---")

report = pd.read_sql("""
SELECT c.Name, p.Product, o.Quantity, o.Total
FROM orders o
JOIN customers c ON o.CustomerID = c.CustomerID
JOIN products p ON o.ProductID = p.ProductID
""", conn)

print(report)

conn.close()

print("\n===== ETL PIPELINE COMPLETED =====")

```

Output:

===== PRACTICAL 9: END-TO-END ETL PIPELINE =====

--- 1. Historical Data ---

Customers:

	CustomerID	Name
0	1	Amit
1	2	Neha
2	3	Rahul

Products:

	ProductID	Product	Price
0	101	Laptop	50000
1	102	Phone	30000
2	103	Headphones	2000

Orders:

	OrderID	CustomerID	ProductID	Quantity
0	1	1	101	1
1	2	2	102	2
2	3	3	103	3

Products:

	ProductID	Product	Price
0	101	Laptop	50000
1	102	Phone	30000
2	103	Headphones	2000

Orders:

	OrderID	CustomerID	ProductID	Quantity
0	1	1	101	1
1	2	2	102	2
2	3	3	103	3

Payments:

	PaymentID	OrderID	Amount
0	1	1	50000
1	2	2	60000
2	3	3	6000

--- 2. Data Cleaning and Transformation ---

	OrderID	CustomerID	ProductID	Quantity	Total
0	1	1	101	1	50000
1	2	2	102	2	60000
2	3	3	103	3	6000

--- 3. Newly Arriving Data ---

	OrderID	CustomerID	ProductID	Quantity	Total
0	4	1	102	1	30000

--- 4. Combined Data ---

	OrderID	CustomerID	ProductID	Quantity	Total
0	1	1	101	1	50000
1	2	2	102	2	60000
2	3	3	103	3	6000
3	4	1	102	1	30000

--- 5. SQL Database Loading ---

Customers, Products, Orders and Payments loaded successfully.

--- 6. Final E-Commerce Report ---

	Name	Product	Quantity	Total
0	Amit	Laptop	1	50000
1	Neha	Phone	2	60000
2	Rahul	Headphones	3	6000
3	Amit	Phone	1	30000

===== ETL PIPELINE COMPLETED =====